CYBERSECURITY TRACK • PYTHON MODULE

Assignment 9

File Integrity Monitoring (FIM) Tool

Done by: Abdulhadi Al-Ahdal

1. Project Overview

The objective of this assignment is to develop a Python tool utilized in Incident Response fields[cite: 3]. It aims to monitor sensitive system files and alert administrators immediately upon any unauthorized modification to their contents by periodically recalculating and verifying hashes[cite: 3].

2. Development Steps

1 SHA-256 Reader Function

Write a function to read a specified file and compute the SHA-256 hash of its contents[cite: 3].

2 Baseline Hash Initialization

Establish the initial baseline hash for the file upon the first execution of the tool[cite: 3].

3 Continuous Monitoring Loop

4 Critical Alert Trigger

Build a continuous loop (`while True`) to recalculate the hash at fixed time intervals (e.g., every 3 seconds) [cite: 3].

Compare current and baseline hashes, triggering an immediate critical security alert upon mismatch[cite: 3].

3. Technical Features & Utilities

Timing Control: Use `time.sleep()` to regulate scan intervals and prevent high CPU utilization[cite: 3].

Chunked Binary Reading: Read large files in binary mode (`rb`) across chunks to ensure memory efficiency[cite: 3].

4. Implementation Source Code

`fim_monitor.py`

Python 3.x

```
import hashlib
import time
import os
import sys

def calculate_file_hash(filepath, chunk_size=65536):
    sha256_hash = hashlib.sha256()
    try:
        # Read file in binary chunks for efficiency
        with open(filepath, "rb") as f:
            for byte_block in iter(lambda: f.read(chunk_size), b""):
                sha256_hash.update(byte_block)
```

```
        return sha256_hash.hexdigest()
    except FileNotFoundError:
        return None

def monitor_file(target_file, interval=3):
    if not os.path.exists(target_file):
        with open(target_file, "w") as f:
            f.write("INITIAL_SENSITIVE_CONFIG_DATA=OK\n")

    print(f"[*] Establishing initial baseline hash for: {target_file}")
    baseline_hash = calculate_file_hash(target_file)
    print(f"[+] Baseline Hash: {baseline_hash}")
    print(f"[*] Starting continuous integrity monitoring loop (interval: {interval}s)... Press Ctrl+C to

    try:
        while True:
            time.sleep(interval)
            current_hash = calculate_file_hash(target_file)

            if current_hash is None:
                print(f"[-] WARNING: Monitored file '{target_file}' was deleted!")
                break

            if current_hash != baseline_hash:
                print("\n" + "!" * 70)
                print(f"[!] CRITICAL SECURITY ALERT: Unauthorized file modification detected!")
                print(f"    Target File    : {target_file}")
                print(f"    Original Hash  : {baseline_hash}")
                print(f"    Tampered Hash  : {current_hash}")
                print(f"    Timestamp      : {time.strftime('%Y-%m-%d %H:%M:%S')}")
                print("!" * 70 + "\n")
```

```
                # Update baseline to prevent alert spam, or break depending on incident response policy
                baseline_hash = current_hash
            else:
                print(f"[*] [{time.strftime('%H:%M:%ST')}] Integrity check passed. No changes detected.")

    except KeyboardInterrupt:
        print("\n[*] FIM monitoring stopped by administrator.")

def main():
    target = sys.argv[1] if len(sys.argv) > 1 else "sensitive_config.txt"
    monitor_file(target, interval=3)

if __name__ == "__main__":
    main()
```

5. Acceptance Criteria

- ✓ **Precise Tamper Detection**
  Ability of the tool to monitor files accurately and trigger an alert immediately upon editing any single character[cite: 3].

- ✓ **Stable Loop Execution**
  Stable, automatic, and error-free operation of the monitoring loop during runtime[cite: 3].